RCA: HMS app Prisma pool cascade — 2026-06-22

Date	2026-06-22
Service	`hms-app` (Next.js 15 monolith, Prisma 6.0.1, tRPC, custom Argon2 session auth)
Environment	dev (EKS namespace `ycare-hms-dev`, RDS `ycare-dev`)
Severity	Multi-hour degraded service; auto-recovery was blocked by structural bugs
Outcome	Resolved by manual pod kill after RDS had already recovered

TL;DR

A routine RDS blip at **04:41 UTC** (most likely a Multi-AZ failover or maintenance reboot) was amplified into a multi-hour incident by **two structural bugs** in the HMS app deployment:

1. **Prisma connection pool sized at 3.** The `DATABASE_URL` has no explicit `connection_limit`. Prisma uses the default `num_physical_cpus × 2 + 1`, and the EKS pod's `cpu: 1500m` limit gets rounded by the Linux cgroup to 1 CPU as reported by Node's `os.cpus()`. Pool = 3 is too small for a Next.js tRPC app where one page load fires 4+ parallel queries.
2. **Liveness probe hits `/auth/login`.** That route triggers ~20 Prisma calls per probe hit via the page's `getCurrentUser()` server-side call and the form's `trpcRQ.auth.login` mutation. Under any DB slowness or pool starvation, the probe hangs past its 3s timeout, fails 3× in 90s, and the kubelet kills the pod. New pod, same conditions, repeat.

RDS recovered within minutes on its own. The pool-exhaustion + probe-kill loop persisted until a manual pod restart after the underlying RDS issue had cleared.

Fix priority (smallest blast radius first):

1. Move liveness probe off `/auth/login` to a no-DB route (e.g. `/healthz`). One-line spec change. Breaks the death spiral at the source.
2. Set `connection_limit=10&pool_timeout=30` explicitly in `DATABASE_URL`. Decouples pool sizing from pod CPU.
3. Investigate why `Deployment/dev-ycare-hms` rolled to revision 613 at 10:22 UTC (cause unknown).

Timeline

Time (UTC)	Event	Evidence
~04:41:05	First `ECONNRESET` in app logs (old pod)	§E1
~04:42:42	`ECONNRESET` continues 97s later	§E1
~10:22:18	New ReplicaSet `dev-ycare-hms-84f49f5994` created; revision 613	§E5
~10:23:35	New pod became Ready	§E5
10:54:56	ActivityLogger queues first log on new pod (single query fits in pool)	§E2
10:55:00	AppointmentService "Getting appointments" starts	§E2
10:55:05	`prisma.session.findUnique()` → **P1001** ("Can't reach DB")	§E2
10:55:33	pharmacy-sale-service "Getting pharmacy sales" starts	§E2
10:55:48	Burst of **P2024** across OPDBilling, Patient, Discharge, PharmacySale, HospitalInfo, ProxyBill, session — all `connection_limit: 3, timeout: 10`	§E2

Time (UTC)	Event	Evidence
10:55:48	`ProxyBillRepository` explicit: "Timed out fetching a new connection from the connection pool"	§E2
10:55:49+	App stuck in pool starvation; subsequent requests hang or fail	§E2
(later)	Manual pod kill; new pod (revision 613) is Ready and serving	user confirmation

Evidence

E1 — ECONNRESET signal on old pod

Two `ECONNRESET` events from inside the container (path `/app/node_modules/...` confirms Docker, not host):


```
{
  "context": "initPgBossLogger",
  "error": { "code": "ECONNRESET" },
  "level": "error",
  "message": "PgBoss error",
  "timestamp": "2026-06-22T04:41:05.041Z"
}
```

```
{
  "context": "initPgBossLogger",
  "error": { "code": "ECONNRESET" },
  "level": "error",
  "message": "PgBoss error",
  "timestamp": "2026-06-22T04:42:42.686Z"
}
```


Prisma wraps the underlying TCP error as `P1001`:

```
PrismaClientKnownRequestError:
Invalid `prisma.logs.create()` invocation:
Can't reach database server at `ycare-dev.ciaklsje1ynv.ap-south-1.rds.amazonaws.com:5432`
    at Mn.handleRequestError (/app/node_modules/@prisma/client/runtime/library.js:121:7338)
{ code: 'P1001', clientVersion: '6.0.1' }
```

Both Prisma and pg-boss hit the same endpoint with `ECONNRESET` — independent clients failing the same way points at the endpoint, not at the clients. This is **not** `ECONNREFUSED` (RDS stopped); it's an RST on an established connection, consistent with Multi-AZ failover or a server-side restart. Sustained 97s+ of `ECONNRESET` rules out transient failover (which resolves in <60s).

E2 — P2024 cascade on new pod

After the new pod came up, the error pattern shifted from `ECONNRESET` to `P2024` (pool exhaustion):


```
{
  "context": "apiHandler",
  "error": {
    "clientVersion": "6.0.1",
    "code": "P2024",
```

```
    "meta": {
      "connection_limit": 3,
      "timeout": 10,
      "modelName": "OPDBilling"
    },
    "name": "PrismaClientKnownRequestError"
  },
  "level": "error",
  "message": "Unknown error",
  "timestamp": "2026-06-22T10:55:48.031Z"
}
```

```
{
  "context": "ProxyBillRepository",
  "level": "error",
  "message": "\nInvalid `prisma.proxyBill.findMany()` invocation:\n\n\nTimed out fetching a
new connection from the connection pool. More info: http://pris.ly/d/connection-pool
(Current connection pool timeout: 10, connection limit: 3)",
  "timestamp": "2026-06-22T10:55:48.384Z"
}
```


Models affected in a single second at 10:55:48: OPDBilling, Patient, Discharge, PharmacySale, HospitalInfo, ProxyBill, Session. The proxyBill message makes the failure mode unambiguous: pool exhausted, queries timing out at 10s.

E3 — DATABASE_URL has no explicit pool config


```
postgres://hms_dev_admin:***@ycare-dev.ciaklsje1ynv.ap-south-
1.rds.amazonaws.com:5432/ycare_hms_dev?sslmode=no-verify
```


The only query parameter is `sslmode=no-verify`. **No `connection_limit`, no `pool_timeout`.** The pool defaults come from Prisma's internal logic.

(Password masked. Original was shared in chat for diagnosis; if the conversation log is preserved anywhere, rotate the credential.)

E4 — Prisma pool default formula and cgroup rounding

Prisma's default for `connection_limit` when not in the URL:

```
connection_limit = num_physical_cpus × 2 + 1
```

In a container with `cpu: 1500m`, the Linux cgroup rounds the quota for `os.cpus()` reporting purposes. **Node.js sees 1 CPU, not 1.5.** Therefore: `1 × 2 + 1 = 3`, which matches §E2 exactly.

`cpu` limit	Node sees	Prisma pool
`1000m`	1	3
`1500m`	1	**3** ← this incident
`2000m`	2	5
`4000m`	4	9

E5 — Pod spec (Deployment `dev-ycare-hms`, ReplicaSet `dev-ycare-hms-84f49f5994`)

```yaml
apiVersion: apps/v1
kind: ReplicaSet
metadata:
  creationTimestamp: '2026-06-22T10:22:18Z'
  annotations:
    deployment.kubernetes.io/revision: '613'
spec:
  replicas: 1
  template:
    spec:
      containers:
        - name: ycare-hms
          image: 265849704119.dkr.ecr.ap-south-1.amazonaws.com/ycare-hms:development-5f238cc
          resources:
            limits:
              cpu: 1500m
              memory: 3Gi
            requests:
              cpu: 128m
              memory: 512Mi
          livenessProbe:
            httpGet:
              path: /auth/login
              port: 3000
            initialDelaySeconds: 60
            periodSeconds: 30
            failureThreshold: 3
            timeoutSeconds: 3
          readinessProbe:
            httpGet:
              path: /auth/login    # same DB-dependent route
              port: 3000
            initialDelaySeconds: 60
            periodSeconds: 30
            failureThreshold: 3
            timeoutSeconds: 3
status:
  availableReplicas: 1
  readyReplicas: 1
```

Two observations matter:

1. `cpu: 1500m` → pool = 3 (§E4).
2. `livenessProbe.path: /auth/login` — DB-dependent route (§E6).

E6 — `/auth/login` call chain (DB-dependent, ~20 Prisma calls per hit)

Layer	File	Prisma hits
Page	`src/app/(auth)/auth/login/page.tsx`	0 direct — calls `getCurrentUser()` server-side, which does
Form	`src/app/(dashboard)/common/auth/features/components/login-form.tsx`	0 — submits via `trpcRQ.auth.login.useMutation`
tRPC resolver	`src/lib/trpc/routers/auth.ts:9,13,16`	imports `prisma from @/lib/db`, instantiates `new AuthService(prisma)`, calls `authService.login(input, ctx.isHttps)`

Layer	File	Prisma hits
Service	`src/app/(dashboard)/common/auth/features/auth-service.ts`	~20 — `prisma.user.findUnique`, `prisma.session.findFirst`, `prisma.session.create`, `prisma.$transaction`, etc.

One liveness probe hit ≈ 20 Prisma calls. With a 3-connection pool, the probe alone consumes the entire pool.

Side note: `login-form.tsx` is imported as `from "@common//auth/features/components/login-form"` — double slash. Works via Next/Webpack path normalization; latent bug worth a one-line fix.

What happened — narrative with evidence

Phase 1 — RDS event at ~04:41 UTC (§E1)

RDS initiated a Multi-AZ failover or maintenance reboot. Existing connections from the HMS app's old pod (running 1.5 days) were RST'd.

Both Prisma and pg-boss fail with `ECONNRESET` on the same endpoint. Independent clients failing the same way ⇒ endpoint-side issue, not client-side.

The Prisma pool being full of dead connections is observable in the logs: the *logger* can't write its own failure, because `prisma.logs.create()` is itself failing. This is the "logger writing its own failure" signal from `hms-docs/prompts/hms-app-db-connectivity-debug.md` §SIGNAL EXTRACTION.

Phase 2 — Old pod stuck in pool death (§E1 continued)

ECONNRESET continues 97s later at 04:42:42. Either RDS is still mid-event, or the Prisma pool is still full of dead connections from the original RST and Prisma's reconnect logic isn't keeping up.

The old pod's liveness probe is also pointed at `/auth/login`. If DB slowness persisted, the probe would have started failing too — but the user reports they killed the old pod manually before K8s would have, so the death spiral from §Phase 5 below may not have run to completion on this pod.

Phase 3 — New ReplicaSet rolled at 10:22 UTC (§E5)

`Deployment/dev-ycare-hms` rolled to revision 613 at 10:22:18 UTC, creating a new ReplicaSet. New pod became Ready by 10:23:35.

The user reports "no changes is made." But revision 613 means the pod template changed. Possible triggers:

- CI/CD auto-rollout on a new commit to the dev branch (image tag `development-5f238cc` is a commit hash)
- Rancher auto-update of an annotation or label (the ReplicaSet is managed by `k3s` per the `managedFields` block)
- ECR image re-tagging combined with `imagePullPolicy: Always`

Open question: `kubectl rollout history deployment/dev-ycare-hms -n ycare-hms-dev --revision=613` against revision 612 would show the diff.

Phase 4 — P2024 cascade on new pod at 10:54–10:55 UTC (§E2)

The new pod's Prisma client creates a fresh pool of 3 connections (§E4). For the first minute, low traffic — ActivityLogger writes succeed.

At 10:55:00, traffic picks up. At 10:55:05, the auth session lookup gets `P1001` — likely RDS still in post-recovery state.

By 10:55:48, multiple parallel queries pile up. With pool = 3:

- 1st–3rd queries get connections and run.
- 4th query waits up to 10s for a free connection.
- If any of the first three are slow (post-failover RDS latency, lock waits), the 4th times out → `P2024`.

- Cascade: each `P2024` leaves a user request hanging; new requests pile up faster than the pool drains.

The proxyBill explicit error makes this unambiguous.

Phase 5 — Liveness probe death spiral (would have happened, §E5 + §E6)

With the probe on `/auth/login` (~20 Prisma calls per hit) and the pool stuck at 3:

```
t=0s:   Probe attempt 1 starts
        → getCurrentUser fires, AuthService.login fires
        → all 3 pool slots consumed
        → DB still slow (post-failover latency)
        → calls exceed probe's 3s timeout
        → probe times out, 1/3 failures

t=30s:  Probe attempt 2
        → same shape, 2/3 failures

t=60s:  Probe attempt 3
        → same, 3/3 failures
        → kubelet kills the pod

t=61s:  New pod starts (replica set ensures 1 replica)
        → fresh Prisma client, fresh pool = 3
        → same DB, same load
        → repeat
```

The structural conditions for the spiral are present. We don't have direct evidence it ran to completion in this incident (the user manually killed the old pod before it would have, and the new pod is "Ready and working" now). But the **next** DB blip will reproduce it.

Phase 6 — Manual recovery

User confirmed: "yes, we killed the previous pod and this seems new one. it is ready. and working."

Current "working" state:

- RDS has fully recovered (long past 04:41 UTC; failover/maintenance done).
- Pool = 3 is not currently stressed because traffic is calm.
- ActivityLogger queue has drained.

Structural bugs are not fixed. Next DB blip will reproduce.

Contributing causes — ranked

C1 (highest leverage) — Liveness probe on a DB-dependent route

Evidence: §E5 (`livenessProbe.path: /auth/login`) + §E6 (~20 Prisma calls per probe).

Effect: Converts any DB slowness into a pod-kill loop. This is the bug that turned a 30-second RDS blip into a multi-hour incident.

Fix: Move probe to `/healthz` or a no-DB route. One-line change in the Deployment spec.

C2 — Prisma pool undersized for the workload

Evidence: §E2 (`connection_limit: 3`), §E3 (no explicit config in URL), §E4 (formula + cgroup rounding), §E5 (`cpu: 1500m`).

Effect: Pool = 3 is too small for a Next.js tRPC monolith. Burst load (one page load = session + appointments + bill types + hospital info + pharmacy sales in parallel = 4+ queries) routinely exceeds the pool.

Fix: Add `&connection_limit=10&pool_timeout=30` to the `DATABASE_URL`.

C3 — RDS-side event (the trigger)

Evidence: §E1 (ECONNRESET pattern), timing (04:41 UTC, ~6 hours before new pod came up).

Most likely cause: Multi-AZ failover or maintenance reboot. Resolved by AWS without user intervention.

Status: Resolved. No action required.

C4 (open) — Silent Deployment rollout at 10:22 UTC

Evidence: §E5 (revision 613, `creationTimestamp: '2026-06-22T10:22:18Z'`).

User reports no manual changes, but the Deployment rolled anyway. Likely CI/CD auto-deploy, but unverified.

Action: `kubectl rollout history deployment/dev-ycare-hms -n ycare-hms-dev --revision=613` diff against 612.

Fix roadmap

In priority order, smallest blast radius first:

1. **Move liveness probe** off `/auth/login` to `/healthz` (or similar no-DB route returning 200 unconditionally). One-line change. Breaks the death spiral at the source.
2. **Set explicit pool size** in `DATABASE_URL`: `?sslmode=no-verify&connection_limit=10&pool_timeout=30`. Decouples pool sizing from pod CPU.
3. **Investigate revision 613** — `kubectl rollout history` diff.
4. **(Optional)** Bump pod CPU to `cpu: 2000m`. Gets pool = 5 via Prisma defaults and gives more CPU headroom generally.
5. **(Optional)** Fix the stray double-slash in `login-form.tsx`: `from "@common//auth/features/components/login-form"`.

What this incident is NOT

- **Not a network failure** — SGs, NACLs, route tables were unchanged. No `ENOTFOUND` or `ETIMEDOUT` in logs.
- **Not a credential rotation** — no P1000 auth errors.
- **Not a DNS issue** — RDS endpoint resolves (otherwise we'd see `ENOTFOUND`, not `ECONNRESET`).
- **Not a Prisma 6.0.1 library bug** — no specific regression; all failures explained by pool size + probe target.
- **Not RDS storage-full or `max_connections` reached** — RDS recovered without intervention; would have required manual scaling if it were.
- **Not a per-request Prisma client creation** — confirmed by the singleton import at `src/lib/trpc/routers/auth.ts:9,13,16`.

References

- Runbook applied: `hms-docs/prompts/hms-app-db-connectivity-debug.md`
- Affected service: `hms-app` (Next.js 15 monolith, App Router, Prisma 6.0.1, tRPC, custom Argon2 session auth)
- Namespace: `ycare-hms-dev` (EKS, managed by Rancher; `k3s` scheduler visible in ReplicaSet metadata)
- RDS: `ycare-dev.ciaklsje1ynv.ap-south-1.rds.amazonaws.com:5432`, database `ycare_hms_dev`
- Deployment: `dev-ycare-hms` (rolled to revision 613 at 10:22 UTC, cause unknown)
- ReplicaSet: `dev-ycare-hms-84f49f5994`
- Evidence inline: §E1 ECONNRESET logs · §E2 P2024 logs · §E3 DATABASE_URL · §E4 Prisma pool formula · §E5 pod spec · §E6 /auth/login call chain